RAG over the GDPR: plain retrieval vs cross-encoder reranking

Itikela Bhaskar · 2023111032 · LMA RAG assignment · 19 October 2026 · code, data and all outputs:
github.com/itikelabhaskar/gdpr-rag-rerank

1. Setup

I used the GDPR (the original 2016 text, 88 pages) because it is full of rules with exceptions and negations, which is exactly what the failure categories are about. I chunked it along its own structure: one chunk per article paragraph, per Article 4 definition and per recital, each starting with its article title. That gave 607 chunks (median 80 tokens, max 349, kept under the bi-encoder's 384-token limit). Because of this, a rule and its exception usually sit in different chunks, for example Art. 17(1) "right to erasure" and Art. 17(3) "paragraphs 1 and 2 shall not apply".

Both pipelines use the same chunks, the same LLM and the same prompt. Only the reranking step differs.

part	choice
bi-encoder	`msmarco-MiniLM-L6-cos-v5`, exact FAISS inner-product index (vectors are normalised, so this is cosine)
cross-encoder	`ms-marco-MiniLM-L6-v2`, same size and training data domain as the bi-encoder
LLM	gemma2 9B (Ollama, local), temperature 0. The prompt says: answer only from the passages, cite them, otherwise reply "Not found in the provided context."
pipeline 1	top 10 by cosine, the first 3 go to the LLM → Answer 1
pipeline 2	the same 10 rescored by the cross-encoder, the new top 3 go to the LLM → Answer 2

To know whether a retrieved chunk is actually right, every question lists the facts a full answer needs and which chunks state each fact. I wrote 16 **eval questions** (4 per category) and labelled them before running anything; only these are averaged. I then wrote 24 **stress questions** to hunt for failures, each labelled before it was run. These are only used as examples.

2. Bi-encoder vs cross-encoder

A bi-encoder turns the question and each chunk into separate vectors and compares them with cosine similarity. The chunk vectors are computed once, ahead of time, so a question only needs one encoding and a fast search. The cost is that a whole paragraph is squeezed into one vector before it ever meets the question, so small but important words like "not", "unless" and "except" get lost.

A cross-encoder reads the question and one chunk together as a single input. Self-attention runs across both, so every word of the question can look at every word of the chunk, and the model outputs one relevance score. This is much more precise, but nothing can be precomputed: it has to run once per (question, chunk) pair at question time. That is why it is used to rerank a short list rather than to search.

A small test shows the difference. I took two questions that differ only by "not" and scored them against the rule and the exception. The bi-encoder sees "when does a company *have to* tell people about a data breach?" and "…*not* have to…" as almost the same question (cosine 0.93), and its scores for Art. 34(1) (the duty) and Art. 34(3) (the exceptions) barely move. The cross-encoder switches its preference: the positive question prefers 34(1) (score 1.24 vs −0.53), the "not" question prefers 34(3) (0.82 vs 0.00). For the same test with erasure, though, it preferred the rule Art. 17(1) both times. So it handles negation better, but not reliably.

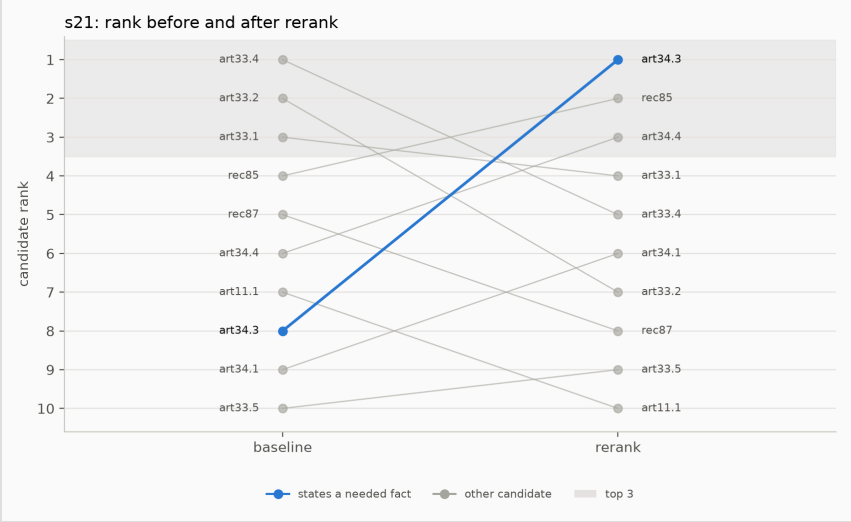
3. Results and rank shifts

eval set (16 questions)	hit@3	facts covered by top 3	all facts in top 3	MRR@10
baseline	0.44	0.40	0.38	0.38
rerank	0.56	0.50	0.44	0.59

Reranking clearly improves ranking: MRR goes from 0.38 to 0.59. When the right chunk is somewhere in the 10, the cross-encoder usually pulls it up. On the 16 eval questions it never reduced the facts available in the top 3, though that did not always turn into better answers. But the bigger limit is recall. On average only 65% of a question's facts are anywhere in the top 10, and 6 of the 16 questions miss a fact completely. Reranking cannot fix those, because it only reorders what the bi-encoder found. Most misses come from plain words not matching legal words: "deleted" never reached Art. 17 "erasure" (its chunks were at rank 23 and lower), and "health information" never reached Art. 9 "data concerning health".

The cases below are where a needed chunk sat at rank 4 to 10 (or the other way round). Each category has at least one case where reranking brought the needed chunk into the top 3. That is a retrieval fix; whether the answer also got better is a separate question, shown in the last column.

question (category)	**needed chunk**	**Answer 1 (baseline)**	**Answer 2 (rerank)**
How quickly must a controller tell the regulator about a breach? (keyword trap, eval)	Art. 33(1) #4 → #1	"Not found"	72-hour rule and reasons if late; misses the "unlikely to cause risk" exception
Do I have to pay a fee to get a copy of my data? (keyword trap)	Art. 12(5) #7 → #2	reasonable fee for extra copies	same, drops "reasonable"; neither says the first copy is free
When is a company *not* required to tell people about a breach? (negation)	Art. 34(3) #8 → #1	"Not found"	all three conditions (encryption, later measures, disproportionate effort); correct
Can EU countries make their own rules about workers' data? (buried)	Art. 88(1) #9 → #1	"Not found"	"Yes", citing Art. 88(1); correct
When does data portability not apply? (exception)	Art. 20(3) #4 → #1	"Not found"	"Not found", even with the chunk at rank 1
Is processing sensitive data like health banned, or are there exceptions? (exception)	Art. 9(2) #2 → #6	"not banned, there are exceptions" (wrong: there is a general ban)	"Not found": reranking pushed the exceptions out



The negation case. Each line is one of the 10 candidates, left = bi-encoder rank, right = rank after reranking. Art. 34(3) (blue) moves from 8 to 1; the grey band is the top 3 that reaches the LLM.

What happened to the answers. The baseline rarely hallucinated. Because the prompt forces it to stay inside the passages, a missing chunk mostly showed up as a refusal or a missing clause. On the eval set the baseline refused 56% of the time and the rerank pipeline 38%. Rerank answers also stated more of the needed facts (35% vs 27%). Fully correct answers were 3 out of 16 for both. Better context also made the model attempt more answers, so it made a few more mistakes (wrong or unsupported in 12.5% vs 6%). The portability case shows that fixing retrieval does not guarantee a better answer: the model still refused with the right chunk at rank 1. The sensitive-data case shows that the reranker can also be wrong.

4. Latency

I timed retrieval on its own (5 warm-ups, then the 16 eval questions × 10 runs, with GPU synchronisation) and full live requests (each eval question once per pipeline, alternating which goes first). Hardware: RTX 4070 laptop GPU for the encoders, FAISS on CPU, gemma2 split about 76% GPU / 24% CPU.

step (mean)	baseline	rerank
query encoding	9.3 ms	9.3 ms
FAISS search over 607 vectors	0.1 ms	0.1 ms
cross-encoder on 10 pairs	–	20.4 ms
retrieval total	9.4 ms	29.8 ms
reranker, measured inside live requests	–	52 ms
full request including the LLM	6.65 s	7.49 s

Reranking roughly triples retrieval time, but retrieval is tens of milliseconds while the LLM takes several seconds. The 0.84 s difference in full requests is mostly not the reranker (about 52 ms of it). The rerank pipeline gave the LLM more useful passages, and it wrote longer answers (on average 50 vs 31 output tokens). Timings varied between runs on this shared laptop, so the exact numbers move a little each run.

5. Why not run the cross-encoder over everything?

Because a cross-encoder score depends on the question, nothing can be computed in advance. Every question needs one pass for every chunk. The bi-encoder encoded all chunks once, before any question existed, so per question it only encodes the question and searches vectors. I measured it directly: scoring all 607 chunks for one question took 0.52 s (about 1,170 pairs per second, batched). That is fine for one document, but the cost grows with the collection for every single question:

chunks	cross-encoder on everything	query encoding + flat vector search
607 (measured)	0.52 s	9 ms
100,000 (extrapolated)	~86 s	~30 ms
1,000,000 (extrapolated)	~14 min	~0.2 s

An approximate index such as HNSW would make the bi-encoder search faster still, since it does not scan every vector. So the sensible design is the one used here: a cheap search over everything to get a short list, then the expensive model on 10 candidates, which costs about 20 ms instead of minutes.

6. Limitations

Sixteen eval questions on one document is a small sample, so these numbers show a direction rather than a general result. The biggest weakness is recall, which reranking cannot help with; a stronger retriever, hybrid keyword and vector search, or a larger candidate pool would be the next things to try. The answer labels were proposed by an AI assistant following written rules, with a second review pass on the disputed ones; they are not independent human grading. The two models are comparable, not identical, so this compares pipelines rather than isolating architecture alone.